CS 360
Programming Languages
Day 32

Final Project

- **If I forgot to ask you for grades, please talk to me briefly after class.**
- Checkpoint 2:
  - Finished lexer/parser, including splices/cuts to produce a clean, readable parse tree
  - The ability to easily produce a parse tree in plain text (ex, **parse-only.txt** as your reader)
  - Functional expander: it should take in a parse tree (or the nodes of a parse tree) and expand it into meaningful, if non-functional, Racket code
  - This includes **one macro/function per parse tree node**
  - Extensive Racket functionality: even if your expander doesn't fully connect with your Racket code, you should have Racket code which does most of what you want

Final Project Support

- If you want an experienced set of **human** eyes on your code, I am happy to oblige.
- I'll give every team 3, 30-minute "tokens" of my time: give me a specific task or file to review, and I'll do my best in 30 minutes.
  - Example: Please review our source->parse-tree pipeline, and clean it up!
  - Example: Please review the syntax patterns / macros in my expander, and clean it up!
  - Example: Please provide a detailed design document for our Racket code. I don't even know where to start!
  - **Don't send me a ton of code that you don't even understand. ;)**
- Or, come to office hours for nearly unlimited time! I'll stay til 6 or 7 pm if I can and you want it! ☺

Review

```
30 rem print 'ignored'
35
50 print "never gets here"
40 end
60 print 'three' : print 1.0 + 3
70 goto 11. + 18.5 + .5 rem ignored
10 print "o" ; "n" ; "e"
20 print : goto 60.0 : end
```

Review

- See Master Recipe
- We have a macro for **b-line** which expands a **b-line** parse tree node to a Racket function **line-x** depending on the line number
- We have a macro which creates **#%module-begin** (always required for any expander!)
- We created a **hash table** mapping *numbers* to *functions* (this is probably not necessary for you)
- We have several techniques for viewing the state of our language: **tokenize-only**, **parse-only**, **shims**, and **the macro stepper**. Use them!!

Advice

- Suppose you have a language which defines pets:
  - **Pet Clod : {type: cat, age: 3, attitude: chill}**

- Suppose your parser has a **pet-def** rule producing (pet-def …) in your parse tree

- The Racket equivalent could be:
  - `(pet “Clod” “cat” 3 “chill”)`, where we impose the arguments in that order through a *contract:*
  - `(define (pet name type age attitude) (list name type age attitude))`

- Then your EXPANDER would need:

```
(define-macro (pet-def DATA ...)
   (pet DATA ...))
```

Warmup:

- Open **expander.rkt**
- What does the **b-line** macro do? Add comments as necessary
- What does the **b-module-begin** macro do? Specifically, what about **with-pattern**? What does the expanded module look like? Be as descriptive and technical as possible!

Exercise:

- Notice: every node of our parse tree has a corresponding macro or function. Go node by node and verify this!

- Explain in a comment what each of the following functions does:
  - **b-rem**
  - **b-print**
  - **b-sum**
  - **b-expr**

- Which macros/functions have we not looked at yet?

Exercise:

- What is **run**? When does it get called?
- Go line by line / function by function. Add comments to help you understand what each part of **run** does.
- For example, we define **line-vec**. What is that?
- Ignore the **with-handlers** for now: just focus on the code they wrap.
- If you don’t know something, look it up (or guess!!)
- Step back, feel the force: what does it look like is going on? Don’t get lost in the parentheses!
- Namely: what is the **for/fold** doing?

Exercise:

- Now for the handlers: what happens if our **line-func** is **b-end** or **b-goto**
- Classic pattern: **exception/handler pattern** (aka **throw/catch** or **raise/except**)
- **Big Hint:** Racket and/or returns the last value of the and/or expression!
- Now try to understand what happens in execution of functions:
  - what changes if we raise a **end-program-signal?**
  - what about a **change-line-signal**?